

UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

Facultad de Ingenieria en Sistemas **1966**

Sistema de Gestion de Lotificadora

Documentacion Tecnica del Proyecto de Programacion en C++

Nombre: Axel Dardon

Carnet: 1690-25-23486

Curso: Programacion 1



1. Introduccion y Alcance

- 1.1 Objetivos del Sistema**
- 1.2 Justificacion Tecnologica**

2. Arquitectura del Sistema

- 2.1 Separacion de Capas**
- 2.2 Patron ORM (Object-Relational Mapping)**
- 2.3 Diagrama de Arquitectura**

3. Diagrama y Modelado de Datos

- 3.1 Esquema Relacional**
- 3.2 Integridad Referencial**
- 3.3 Script de Creacion de la Base de Datos**

4. Guia de Instalacion y Compilacion

- 4.1 Requisitos del Sistema**
- 4.2 Estructura de Directorios**
- 4.3 Comando de Compilacion**

5. Manual de Usuario y Evidencia de Funcionamiento

- 5.1 Pantalla Principal del Sistema**
- 5.2 Modulo de Gestion de Proyectos**
- 5.3 Validacion del Buffer de Entrada**
- 5.4Codigo Fuente del Sistema**

1. Introduccion y Alcance

El presente documento tecnico describe en detalle el Sistema de Gestion de Lotificadora, una aplicacion de consola desarrollada en el lenguaje C++ con conectividad nativa al gestor de bases de datos MySQL 8.0. El sistema implementa un patron de arquitectura basado en *Object-Relational Mapping* (ORM) que abstrae las operaciones de persistencia relacional mediante una capa de modelos orientados a objetos.

La solucion fue concebida como proyecto academico para el curso de Programacion de la Universidad Mariano Galvez de Guatemala, con el proposito de demostrar la integracion entre un lenguaje compilado nativo de alto rendimiento y un sistema de gestion de bases de datos relacional en un entorno real de desarrollo.

1.1 Objetivos del Sistema

El sistema tiene como finalidad automatizar la gestion administrativa de una empresa lotificadora, permitiendo el control sistematico de dos entidades fundamentales:

- ♦ **Proyectos inmobiliarios:** Representan los desarrollos residenciales o comerciales que la empresa comercializa. Cada proyecto agrupa un conjunto de lotes disponibles para venta.
- **Lotes:** Unidades inmobiliarias individuales pertenecientes a un proyecto especifico. Cada lote cuenta con atributos como numero identificador, precio de venta y estado actual (disponible, vendido o reservado).

Los objetivos operacionales del sistema incluyen:

1. Registrar nuevos proyectos inmobiliarios con nombre y ubicacion geografica.
2. Visualizar el listado completo de proyectos activos en el sistema.
3. Modificar la informacion de proyectos existentes.
4. Eliminar proyectos del catalogo con manejo de restricciones de integridad referencial.
5. Gestionar lotes asociados a proyectos mediante un modulo independiente con operaciones CRUD completas.

1.2 Justificacion Tecnologica

La seleccion del stack tecnologico responde a criterios de rendimiento, madurez de la plataforma y aplicabilidad en entornos de desarrollo profesional:

Tabla 1 Comparativa de Tecnologias Seleccionadas

Tecnologia	Rol en el Sistema	Justificacion Tecnica
C++ (ISO 11/14)	Lenguaje de implementacion	Lenguaje compilado nativo que genera codigo maquina de alto rendimiento. Gestión manual de memoria mediante punteros, ideal para aplicaciones de consola con requisitos de latencia minima y footprint reducido.
MySQL 8.0	SGBD relacional	Sistema de gestion de bases de datos de codigo abierto con soporte completo de SQL estandar, transacciones ACID, claves foraneas con politicas CASCADE y motor InnoDB por defecto.
MySQL API	C Conector nativo (libmysqlclient)	Biblioteca oficial de Oracle que proporciona funciones C para establecer conexiones, ejecutar sentencias SQL y recuperar resultados mediante estructuras MYSQL_RES.
MinGW g++	Compilador	Compilador GNU para Windows que soporta el estandar C++11/14, con capacidad de enlazado estatico y dinamico contra bibliotecas externas.

La combinacion de C++ con MySQL mediante su API nativa constituye una arquitectura robusta ampliamente utilizada en sistemas de grado empresarial donde el rendimiento de la capa de persistencia es critico. El patron ORM implementado manualmente demuestra la comprension profunda del mapeo objeto-relacional sin depender de frameworks de terceros.

2. Arquitectura del Sistema

El sistema esta organizado siguiendo un modelo de arquitectura por capas que separa claramente las responsabilidades de presentacion, logica de negocio y persistencia de datos. Este enfoque garantiza la mantenibilidad del codigo y facilita futuras extensiones del sistema.

2.1 Separacion de Capas

La aplicacion se estructura en tres capas funcionales bien definidas:

Tabla 2 Descripcion de Capas Arquitectonicas

Capa	Componentes	Responsabilidad
Presentacion (CLI)	Menus interactivos ANSI, funcion leer_entero()	Renderizar la interfaz de usuario en terminal, capturar entrada del usuario mediante flujo estandar (stdin), validar tipos de datos y presentar resultados formateados con colores.
Logica de Negocio (ORM)	Clases Proyecto , Lote , EloquentORM	Encapsular las reglas del dominio, mapear objetos a registros relacionales y exponer metodos de alto nivel (save() , update() , remove()) que abstraen las sentencias SQL.
Persistencia (MySQL)	API C de MySQL, conexion MYSQ L*, resultados MYSQL_RES*	Establecer y gestionar la conexion TCP/IP con el servidor MySQL, ejecutar sentencias SQL preparadas y mapear resultados tabulares a estructuras de datos en memoria.

2.1.1 Capa de Presentacion - Interfaz de Linea de Comandos (CLI)

La interfaz de usuario se implementa mediante una consola interactiva que utiliza secuencias de escape ANSI para la coloracion del texto. El sistema presenta un menu principal jerarquico que permite la navegacion entre modulos funcionales. Los colores aplicados incluyen:

- ♦ Color dorado/amarillo: Titulos de seccion y encabezados de modulos.

- ♦ **Color blanco/cian:** Opciones de menu y texto informativo.
- ♦ **Lineas de separacion:** Caracteres ASCII (=) que delimitan visualmente las secciones.

2.2 Patron ORM (Object-Relational Mapping)

El *Object-Relational Mapping* es un patron de diseño de software que permite interactuar con una base de datos relacional utilizando paradigmas de programacion orientada a objetos, eliminando la necesidad de escribir sentencias SQL directamente en el codigo de la aplicacion.

En este proyecto, el ORM se implemento manualmente mediante la clase base abstracta `EloquentORM`, que define la interfaz comun para todas las entidades persistentes. Las clases concretas `Proyecto` y `Lote` heredan de esta base y especializan su comportamiento.

2.2.1 Clase EloquentORM - Interfaz Base

La clase `EloquentORM` establece el contrato que deben cumplir todas las entidades del sistema mediante metodos virtuales puros:

```
class EloquentORM {
public:
    // Persistencia
    virtual bool save() = 0;
    virtual bool update() = 0;
    virtual bool remove() = 0;

    // Recuperacion
    virtual void find(int id) = 0;
    virtual void all() = 0;

    // Presentacion
    virtual void show() = 0;
};
```

2.2.2 Abstraccion de Sentencias SQL

El patron ORM traduce las operaciones orientadas a objetos en sentencias SQL estandar. La siguiente tabla detalla la correspondencia entre los metodos del ORM y las sentencias SQL generadas:

Tabla 3 Mapeo Metodos ORM a Sentencias SQL

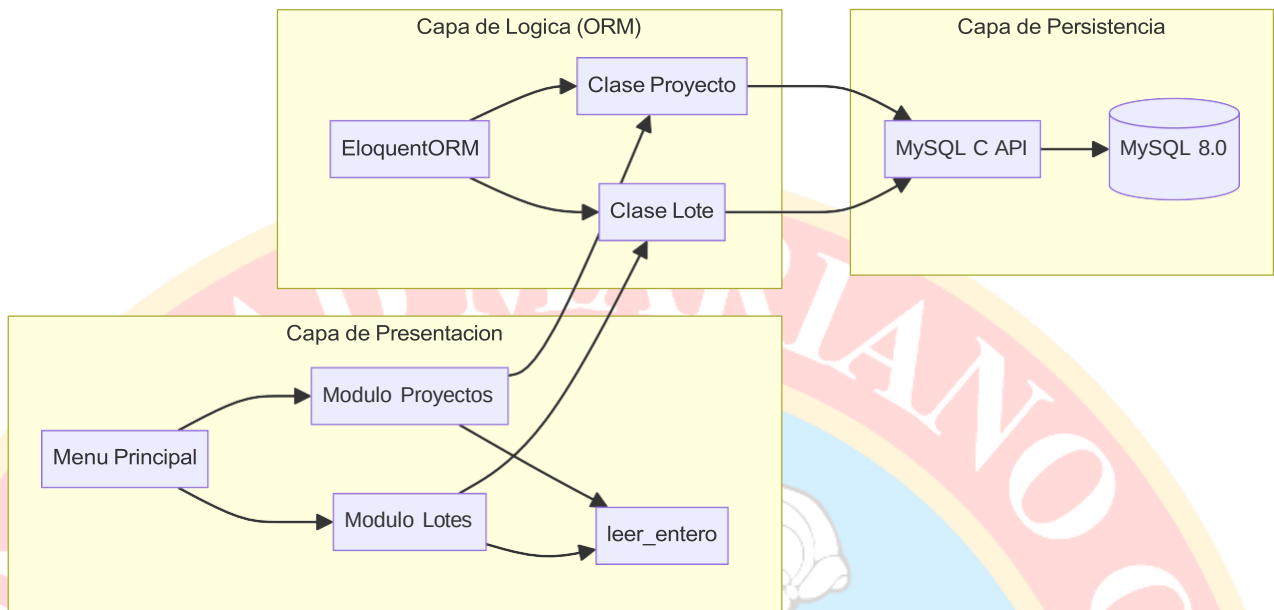
Metodo ORM	Sentencia SQL Generada	Descripcion
<code>obj.save()</code>	<code>INSERT INTO tabla (col1, col2) VALUES (?, ?)</code>	Crea un nuevo registro en la tabla correspondiente con los valores almacenados en los atributos del objeto. Retorna el ID autogenerado.
<code>obj.update()</code>	<code>UPDATE tabla SET col1=?, col2=? WHERE id=?</code>	Modifica los campos del registro existente identificado por la clave primaria del objeto.
<code>obj.remove()</code>	<code>DELETE FROM tabla WHERE id=?</code>	Elimina el registro identificado por la clave primaria. En caso de existir restriccion de integridad referencial, el SGBD aplica la politica configurada (CASCADE).
<code>obj.find(id)</code>	<code>SELECT * FROM tabla WHERE id=?</code>	Recupera un unico registro por su clave primaria y carga sus valores en los atributos del objeto.
<code>obj.all()</code>	<code>SELECT * FROM tabla</code>	Recupera el conjunto completo de registros de la tabla para su visualizacion en forma tabular.

Ventajas del Patron ORM

La abstraccion ORM elimina la duplicacion de codigo SQL, centraliza la logica de persistencia en las clases de modelo y reduce el acoplamiento entre la capa de presentacion y el esquema de base de datos. Cualquier modificacion en la estructura relacional solo requiere actualizar la capa de mapeo, sin afectar los modulos de interfaz.

2.3 Diagrama de Arquitectura

El siguiente diagrama ilustra la interaccion entre las capas del sistema y el flujo de datos desde la interfaz de usuario hasta el gestor de base de datos:



El flujo de ejecución sigue un patron sincrono: el usuario selecciona una opcion del menu, la capa de presentacion invoca el metodo correspondiente en el modelo ORM, y este traduce la operacion a una sentencia SQL que se ejecuta a través del conector nativo de MySQL.

3. Diagrama y Modelado de Datos

El esquema de base de datos se diseño bajo los principios del modelo relacional, garantizando la integridad de los datos mediante el uso de claves primarias, claves foraneas y restricciones de dominio. La base de datos tabl notificadora_db contiene dos principales que establecen una relacion maestro-detalle.

3.1 Esquema Relacional

Tabla 4 Entidad: proyectos

Columna	Tipo de Dato	Restricciones	Descripcion
id	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	Identificador unico del proyecto. Se genera automaticamente por el SGBD.
nombre_proyecto	VARCHAR(100)	NOT NULL	Nombre descriptivo del desarrollo inmobiliario (ej. "Residenciales Tikal").
ubicacion	VARCHAR(100)	NOT NULL	Direccion o zona geografica donde se ubica el proyecto.

Tabla 5 Entidad: lotes

Columna	Tipo de Dato	Restricciones	Descripcion
id	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	Identificador unico del lote. Generado automaticamente.
numero_lote	VARCHAR(20)	NOT NULL	Identificador alfanumerico del lote (ej. "L-001").
precio	DECIMAL(10,2)	NOT NULL	Precio de venta del lote con dos decimales de precision.
estado	VARCHAR(20)	NOT NULL, DEFAULT 'disponible'	Situacion actual: disponible, vendido o reservado.
proyecto_id	INT	FOREIGN KEY, NOT NULL	Llave foranea que referencia al proyecto al que pertenece el lote.

3.2 Integridad Referencial

La relacion entre las entidades `proyectos` y `lotes` se establece mediante una llave foranea denominada `proyecto_id` en la tabla `lotes`. Esta columna referencia la llave primaria `id` de la tabla `proyectos`, estableciendo una cardinalidad de uno-a-muchos: un proyecto puede tener multiples lotes asociados, pero cada lote pertenece exactamente a un proyecto.

El script SQL configura las siguientes politicas de integridad referencial mediante las clausulas `ON DELETE CASCADE` y `ON UPDATE CASCADE`:

- ♦ **ON DELETE CASCADE:** Si un proyecto se elimina de la tabla `proyectos`, todos los lotes asociados a ese proyecto se eliminan automaticamente de la tabla `lotes`. Esto garantiza que no existan registros huérfanos en la base de datos.
- ♦ **ON UPDATE CASCADE:** Si el valor de la clave primaria `id` de un proyecto se

modifica (escenario poco frecuente con campos autoincrementables), la llave foranea



`proyecto_id` en la tabla `lotes` se actualiza en cascada para mantener la consistencia referencial.

3.3 Script de Creacion de la Base de Datos

El archivo `script.sql` contiene la definicion completa del esquema relacional, incluyendo la creacion de la base de datos, la definicion de tablas con sus restricciones, y datos de prueba para la verificacion del sistema:

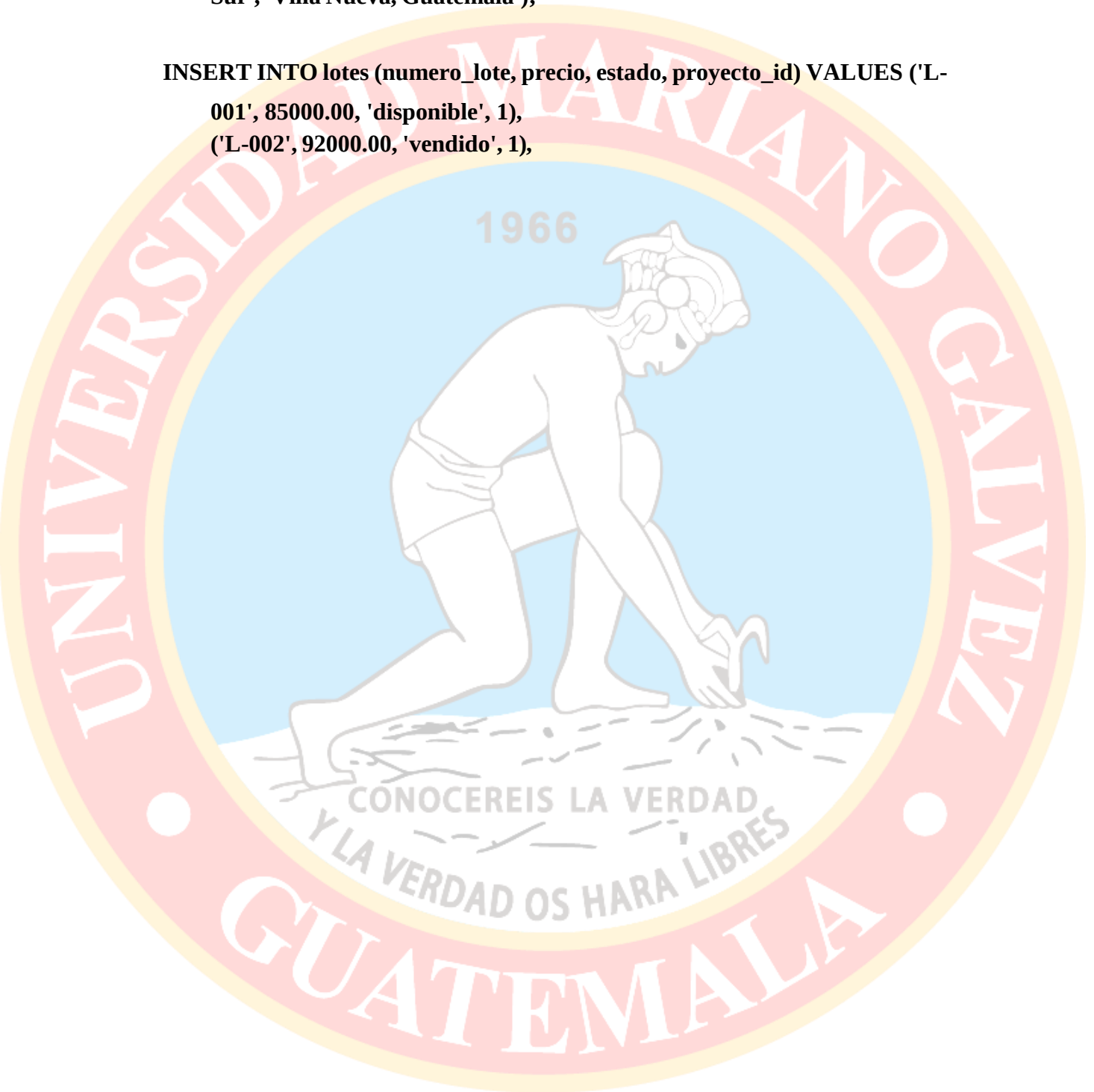


```
-- =====  
-- SCRIPT SQL: Sistema de Lotificadora  
-- Universidad Mariano Galvez de Guatemala  
-- Curso: Programacion en C++  
-- =====  
  
-- Paso 1: Crear la base de datos si no existe CREATE  
DATABASE IF NOT EXISTS lotificadora_db  
    CHARACTER SET utf8  
    COLLATE utf8_general_ci;  
  
USE lotificadora_db;  
  
-- Paso 2: Tabla PROYECTOS  
CREATE TABLE IF NOT EXISTS proyectos (  
    id INT NOT NULL AUTO_INCREMENT,  
    nombre_proyecto VARCHAR(100) NOT  
    NULL, ubicacion VARCHAR(100) NOT  
    NULL, PRIMARY KEY (id)  
);  
  
-- Paso 3: Tabla LOTES  
CREATE TABLE IF NOT EXISTS lotes (  
    id INT NOT NULL AUTO_INCREMENT,  
    numero_lote VARCHAR(20) NOT  
    NULL, precio DECIMAL(10, 2) NOT  
    NULL,  
    estado VARCHAR(20) NOT NULL DEFAULT  
    'disponible', proyecto_id INT NOT NULL,  
    PRIMARY KEY (id),  
    CONSTRAINT fk_lote_proyecto  
        FOREIGN KEY (proyecto_id)  
        REFERENCES proyectos (id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```


-- Paso 4: Datos de Prueba

```
INSERT INTO proyectos (nombre_proyecto, ubicacion) VALUES  
('Residenciales Tikal', 'Zona 7, Ciudad de Guatemala'), ('Lotificadora  
Peten', 'Santa Elena, Peten'),  
('Villas del Lago', 'Amatitlan, Guatemala'), ('Residenciales del  
Sur', 'Villa Nueva, Guatemala');
```

```
INSERT INTO lotes (numero_lote, precio, estado, proyecto_id) VALUES ('L-  
001', 85000.00, 'disponible', 1),  
('L-002', 92000.00, 'vendido', 1),
```



```

-- Creación de la base de datos
CREATE DATABASE Proyecto;

-- Uso de la base de datos
USE Proyecto;

-- Creación de la tabla de usuarios
CREATE TABLE usuarios (
    id_usuario INT(4) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    fecha_registro DATE NOT NULL,
    fecha_actualizacion DATE NOT NULL,
    estado ENUM('activo', 'inactivo') NOT NULL,
    rol ENUM('usuario', 'administrador') NOT NULL
);

-- Creación de la tabla de productos
CREATE TABLE productos (
    id_producto INT(4) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    descripcion VARCHAR(255) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    stock INT(4) UNSIGNED NOT NULL,
    fecha_registro DATE NOT NULL,
    fecha_actualizacion DATE NOT NULL,
    estado ENUM('activo', 'inactivo') NOT NULL
);

-- Creación de la tabla de pedidos
CREATE TABLE pedidos (
    id_pedido INT(4) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT(4) UNSIGNED NOT NULL,
    id_producto INT(4) UNSIGNED NOT NULL,
    cantidad INT(4) UNSIGNED NOT NULL,
    fecha_registro DATE NOT NULL,
    fecha_actualizacion DATE NOT NULL,
    estado ENUM('pendiente', 'procesado', 'cancelado') NOT NULL
);

-- Creación de la tabla de pagos
CREATE TABLE pagos (
    id_pago INT(4) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    id_pedido INT(4) UNSIGNED NOT NULL,
    monto DECIMAL(10,2) NOT NULL,
    fecha_registro DATE NOT NULL,
    fecha_actualizacion DATE NOT NULL,
    estado ENUM('pendiente', 'procesado', 'cancelado') NOT NULL
);

-- Inserción de datos de prueba
INSERT INTO usuarios (nombre, apellido, correo, fecha_registro, fecha_actualizacion, estado, rol) VALUES
('Juan', 'Pérez', 'juan.perez@example.com', '2023-01-01', '2023-01-01', 'activo', 'usuario'),
('María', 'García', 'maria.garcia@example.com', '2023-01-02', '2023-01-02', 'activo', 'usuario'),
('Carlos', 'Rodríguez', 'carlos.rodriguez@example.com', '2023-01-03', '2023-01-03', 'activo', 'administrador'),
('Ana', 'López', 'ana.lopez@example.com', '2023-01-04', '2023-01-04', 'inactivo', 'usuario'),
('Pedro', 'Martínez', 'pedro.martinez@example.com', '2023-01-05', '2023-01-05', 'activo', 'usuario');

INSERT INTO productos (nombre, descripcion, precio, stock, fecha_registro, fecha_actualizacion, estado) VALUES
('Camisa', 'Camisa de algodón', 15.99, 100, '2023-01-01', '2023-01-01', 'activo'),
('Pantalón', 'Pantalón de mezclilla', 29.99, 50, '2023-01-02', '2023-01-02', 'activo'),
('Zapatos', 'Zapatos de cuero', 89.99, 20, '2023-01-03', '2023-01-03', 'activo'),
('Vestido', 'Vestido de seda', 49.99, 10, '2023-01-04', '2023-01-04', 'activo'),
('Sombrero', 'Sombrero de paja', 19.99, 30, '2023-01-05', '2023-01-05', 'activo');

INSERT INTO pedidos (id_usuario, id_producto, cantidad, fecha_registro, fecha_actualizacion, estado) VALUES
(1, 1, 2, '2023-01-01', '2023-01-01', 'pendiente'),
(2, 2, 1, '2023-01-02', '2023-01-02', 'procesado'),
(3, 3, 1, '2023-01-03', '2023-01-03', 'cancelado'),
(4, 4, 1, '2023-01-04', '2023-01-04', 'pendiente'),
(5, 5, 1, '2023-01-05', '2023-01-05', 'procesado');

INSERT INTO pagos (id_pedido, monto, fecha_registro, fecha_actualizacion, estado) VALUES
(1, 31.98, '2023-01-01', '2023-01-01', 'pendiente'),
(2, 29.99, '2023-01-02', '2023-01-02', 'procesado'),
(3, 89.99, '2023-01-03', '2023-01-03', 'cancelado'),
(4, 49.99, '2023-01-04', '2023-01-04', 'pendiente'),
(5, 19.99, '2023-01-05', '2023-01-05', 'procesado');
    
```

Figura 1 Captura del script SQL de creación de la base de datos, mostrando la estructura completa con tablas, restricciones y datos de prueba.

4. Guía de Instalación y Compilación

El siguiente procedimiento detalla los pasos necesarios para configurar el entorno de desarrollo, compilar el proyecto desde código fuente y ejecutar la aplicación en un sistema Windows con la suite de herramientas MinGW.

4.1 Requisitos del Sistema

Tabla 6 Requisitos de Hardware y Software

Componente	Requisito Mínimo	Observaciones
Sistema Operativo	Windows 10/11 (64-bit)	Compatible con Windows 7 SP1 con actualizaciones recientes.
Memoria RAM	4 GB	Recomendado 8 GB para entornos con MySQL local.
Compilador	MinGW-w64 con g++ 8.1+	Debe soportar el estándar C++11 como mínimo.
Base de Datos	MySQL Server 8.0	MySQL Community Edition o XAMPP/WAMP.
MySQL C API	libmysqlclient (64-bit)	Bibliotecas libmysql.lib y headers mysql.h.

4.2 Estructura de Directorios

El proyecto debe organizarse en la siguiente estructura de carpetas antes de la compilación:

```

lotificadora/
|
|-- include/                # Headers de MySQL C API
|   |-- mysql.h
|   |-- mysql_com.h
|   |-- mysql_time.h
|   |-- mysql_version.h
|   |-- my_list.h
|   |-- my_alloc.h
|   |-- typelib.h
|   |-- my_debug.h
|   |-- errmsg.h
|   |-- my_global.h
|   |-- my_sys.h
|   |-- mctype.h
|   |-- mstring.h
|   |-- my_xml.h
|   |-- my_pthread.h
|   |-- decimal.h
|   |-- my_attribute.h
|   |-- my_dir.h
|   |-- hash.h
|   |-- field_types.h
|   |-- my_command.h
|   |-- client_plugin.h
|   |-- plugin_auth_common.h
|   |-- udf_registration_types.h
|
|-- lib/                    # Bibliotecas estaticas/dinamicas de MySQL
|   |-- libmysql.lib
|   |-- libmysql.dll
|
|-- main.cpp                # Código fuente principal del sistema
|-- script.sql              # Script de creación de la base de datos
|-- lotificadora.exe        # Ejecutable generado (post-compilación)

```

4.3 Comando de Compilación

La compilación se realiza mediante el compilador `g++` de MinGW desde la línea de comandos (PowerShell o CMD). El comando incluye las siguientes opciones:

- ♦ `-std=c++11` : Especifica el estándar de lenguaje C++11 para el soporte de características modernas.

- ♦ `-I include` : Indica la ruta al directorio que contiene los archivos de cabecera (`.h`) de la API de MySQL.
- ♦ `-L lib` : Indica la ruta al directorio que contiene la biblioteca estática `libmysql.lib` .
- ♦ `-l mysql` : Especifica el nombre de la biblioteca a enlazar (sin el prefijo "lib" ni la extensión).
- ♦ `-o lotificadora` : Define el nombre del archivo ejecutable de salida.

El comando completo de compilación es:

```
g++ -std=c++11 main.cpp -o lotificadora -I include -L lib -l mysql
```

Nota Importante

Antes de ejecutar el programa compilado, asegúrese de que el archivo `libmysql.dll` se encuentre en el mismo directorio que el ejecutable `lotificadora.exe` , o que su ubicación esté registrada en la variable de entorno `PATH` del sistema. De lo contrario, Windows no podrá resolver las dependencias dinámicas y el programa fallará al iniciar.

4.3.1 Verificación de la Conexión

Al ejecutar el programa, el sistema establece una conexión con el servidor MySQL utilizando los siguientes parámetros de conexión (configurables en el código fuente):

Tabla 7 Parametros de Conexión a MySQL

Parametro	Valor por Defecto	Descripción
Host	localhost	Dirección del servidor MySQL (127.0.0.1)
Usuario	root	Cuenta de administrador de MySQL
Contraseña	(vacía)	Autenticación sin contraseña (entorno de desarrollo)
Puerto	3306	Puerto TCP predeterminado de MySQL Server
Base de Datos	lotificadora_d b	Esquema creado mediante el script SQL

La aplicación verifica el estado de la conexión en el momento de la inicialización y muestra un mensaje de confirmación en la consola cuando la conexión se establece exitosamente.

5. Manual de Usuario y Evidencia de Funcionamiento

Esta seccion documenta el funcionamiento operativo del sistema mediante la descripcion detallada de sus interfaces de usuario y las capturas de pantalla que evidencian su correcto comportamiento en tiempo de ejecucion.

5.1 Pantalla Principal del Sistema

Al iniciar la aplicacion, el sistema presenta el Menu Principal con un encabezado decorativo que identifica el nombre del sistema y la institucion. La interfaz utiliza caracteres ASCII de linea doble para delimitar visualmente el area del menu y colores ANSI para diferenciar los elementos interactivos:

```
=====
SISTEMA DE LOTIFICADORA (UMG)
=====
1. Modulo - Proyectos
2. Modulo - Lotes
0. Salir del sistema
=====
Seleccione: |
```

Figura 2 Menu principal del Sistema de Lotificadora mostrando las opciones de navegacion entre modulos.

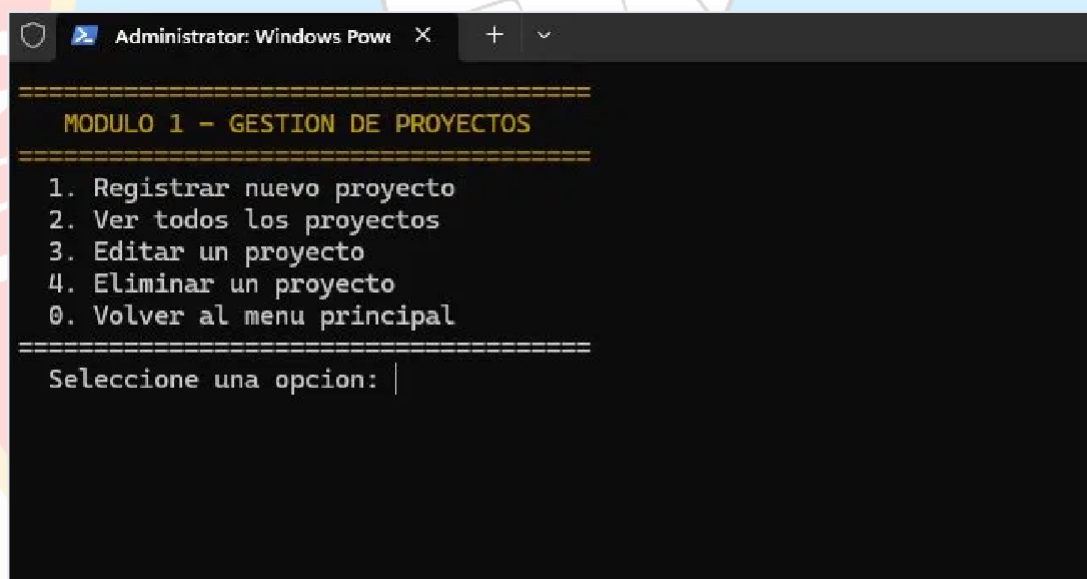
Las opciones disponibles en el menu principal son:

Tabla 8 Opciones del Menu Principal

Opcion	Descripcion	Accion
1	Modulo Proyectos	- Accede al submenu de gestion de proyectos inmobiliarios.
2	Modulo - Lotes	Accede al submenu de gestion de lotes inmobiliarios.
0	Salir del sistema	Termina la ejecucion del programa y cierra la conexion con MySQL.

5.2 Modulo de Gestion de Proyectos

Al seleccionar la opcion 1 del menu principal, el sistema despliega el submenu de Gestion de Proyectos, que proporciona operaciones CRUD (Create, Read, Update, Delete) completas sobre la entidad `proyectos` :



```

Administrator: Windows PowerShell
=====
MODULO 1 - GESTION DE PROYECTOS
=====
1. Registrar nuevo proyecto
2. Ver todos los proyectos
3. Editar un proyecto
4. Eliminar un proyecto
0. Volver al menu principal
=====
Seleccione una opcion: |
  
```

Figura 3 Submenu del Modulo 1 - Gestion de Proyectos con las operaciones CRUD disponibles.

Las funciones disponibles en este modulo son:

1. Registrar nuevo proyecto: Solicita al usuario el nombre del proyecto y su ubicacion. Crea un nuevo registro en la base de datos mediante el metodo `save()` del ORM.

2. **Ver todos los proyectos:** Recupera y muestra en formato tabular el listado completo de proyectos registrados, incluyendo su ID, nombre y ubicacion.
3. **Editar un proyecto:** Solicita el ID del proyecto a modificar, permite actualizar sus campos y aplica los cambios mediante el metodo `update()` del ORM.
4. **Eliminar un proyecto:** Solicita confirmacion antes de eliminar el registro. Debido a la restriccion `ON DELETE CASCADE`, la eliminacion de un proyecto tambien elimina todos sus lotes asociados.
5. **Volver al menu principal:** Regresa a la pantalla de inicio del sistema.

El modulo de Lotes (opcion 2 del menu principal) presenta una estructura de submenu identica, adaptada para la gestion de la entidad `lotes`, incluyendo la seleccion del proyecto al que pertenece cada lote mediante su `proyecto_id`.

5.3 Validacion del Buffer de Entrada

El sistema implementa una funcion auxiliar denominada `leer_entero()` que garantiza la integridad de la entrada de datos numerica por parte del usuario. Esta funcion resuelve un problema comun en aplicaciones de consola C++: la corrupcion del flujo de entrada (`std::cin`) cuando el usuario introduce valores no numericos.

El funcionamiento de `leer_entero()` es el siguiente:

1. Lee una linea completa de texto desde `std::cin` utilizando `std::getline()`.
2. Intenta convertir la cadena leida a un valor entero mediante `std::stoi()`.
3. Si la conversion es exitosa, retorna el valor entero validado.
4. Si la conversion falla (por ejemplo, el usuario ingreso letras), captura la excepcion `std::invalid_argument` y solicita nuevamente la entrada, mostrando un mensaje de error descriptivo.

```
int leer_entero(const string& mensaje) {
    string entrada;
    int valor;
    while (true) {
        cout << mensaje;
        getline(cin, entrada);
        try {
            valor = stoi(entrada);
            return valor;
        } catch (invalid_argument&) {
            cout << "Entrada no valida. Intente de nuevo.\n";
        }
    }
}
```

Esta tecnica de validacion previene los bucles infinitos causados por el estado de error del flujo `cin` y proporciona una experiencia de usuario robusta ante entradas incorrectas.

5.4 Codigo Fuente del Sistema

La siguiente captura muestra el codigo fuente completo del sistema tal como se implemento en el archivo `main.cpp` . Se aprecia la estructura modular con las clases del ORM, las implementaciones de los modelos Proyecto y Lote, las funciones auxiliares y el punto de entrada `main()` :



Figura 4 Vista completa del código fuente en C++ mostrando las clases ORM, modelos de datos y lógica de menús.

El código fuente se organiza en las siguientes secciones lógicas:

- ◆ **Inclusiones y configuración:** Headers estándar de C++ y la biblioteca `mysql.h`.
- ◆ **Clase EloquentORM:** Definición de la interfaz base abstracta del ORM.
- ◆ **Clase Proyecto:** Modelo de la entidad proyectos con atributos, constructor y métodos CRUD.
- ◆ **Clase Lote:** Modelo de la entidad lotes con atributos, constructor y métodos CRUD.
- ◆ **Funciones auxiliares:** Implementación de `leer_entero()` y utilidades de presentación.
- ◆ **Funciones de menú:** Implementaciones de lógica `menu_proyectos()` y `menu_lotes()` con de navegación.